



LV08 재귀함수 가지치기

https://youtu.be/BkmqvLL_5cs?feature=shared

<https://youtu.be/roEdb3yMKLY?feature=shared>

재귀함수 트리구조 가지치기(Pruning)

가지치기란?

기본적인 재귀 함수 호출 시 특정 return 조건을 추가하여 진입하지 않을 조합을 줄이고, 인위적으로 막아주는 것이 있다.

🌳 나무의 가지를 자르는 것처럼, 불필요한 재귀 호출을 미리 차단하여 효율성을 높이는 기법

다양한 ABC 세종류의 카드를 가지고 있다. 이 중 3장을 뽑을 때 모든 조합을 출력하라.

단 A로 시작하는 조합은 제외 시킨다.



```

시작
 / | \
A  B  C (Level 1)
 /\ /\ /\
X ... ... (Level 2) ← A로 시작하는 가지를 X로 차단
 /\ /\ /\
... ... ... (Level 3)

```

가지치기 방법1: 진입 차단 방식 (Prevention)

🚫 애초에 들어가지도 않게 막는 방법

- `continue` 를 사용해서 반복문에서 해당 경우를 건너뛴다
- **장점:** 메모리와 시간을 아예 사용하지 않음
- **단점:** 조건이 복잡해질 수 있음

```
#include <iostream>
```

```
char path[5] = "";
```

```
void test(int level)
```

```
{
```

```
// 🎯 목표 깊이에 도달하면 결과 출력
```

```

if (level == 3)
{
    std::cout << path << std::endl;
    return;
}

for (size_t i = 0; i < 3; i++)
{
    // 💡 핵심: Level 0에서 'A'(i==0)인 경우 아예 건너뛰기
    if (level == 0 && ('A' + i) == 'A')
        continue; // 다음 i로 넘어감 (B, C만 시도)

    path[level] = 'A' + i; // 현재 레벨에 문자 저장
    test(level + 1);      // 다음 레벨로 재귀 호출
    path[level] = 0;      // 백트래킹: 원상복구
}
}

```

☁ 실행 과정 예시:

```

Level 0: A는 continue로 건너뛰고, B와 C만 시도
├─ B 선택 → test(1) 호출
│   └─ BA, BB, BC 생성
└─ C 선택 → test(1) 호출
    └─ CA, CB, CC 생성

```

가지치기 방법2: 조기 종료 방식 (Early Return)

👉 일단 들어가긴 했는데, 조건 확인 후 바로 나가는 방법

- `return` 을 사용해서 함수를 즉시 종료
- **장점:** 복잡한 조건도 함수 내에서 처리 가능
- **단점:** 이미 함수에 진입했으므로 약간의 오버헤드 존재

```
#include <iostream>
```

```

char path[5] = "";
void test(int level)

```

```

{
    // 💡 핵심: 연속된 같은 문자가 2개 나오면 즉시 종료
    // level-2와 level-1 위치의 문자가 같으면 패턴 위반
    if (level >= 2 && path[level - 2] == path[level - 1])
        return; // 함수 즉시 종료 (더 이상 진행 안함)

    if (level == 3)
    {
        std::cout << path << std::endl;
        return;
    }

    for (size_t i = 0; i < 3; i++)
    {
        path[level] = 'A' + i;
        test(level + 1); // 다음 레벨에서 조건 검사
        path[level] = 0;
    }
}

```

☁ 실행 과정 예시:

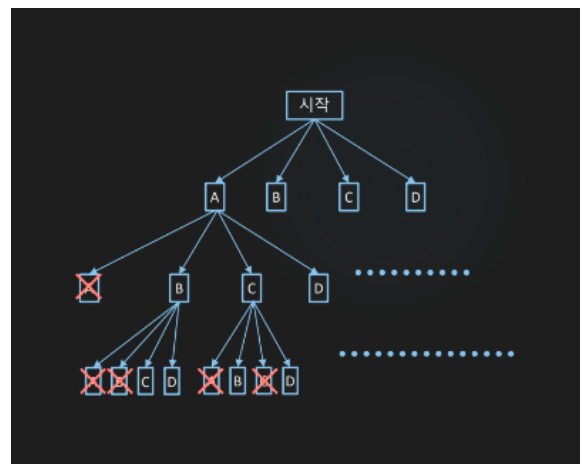
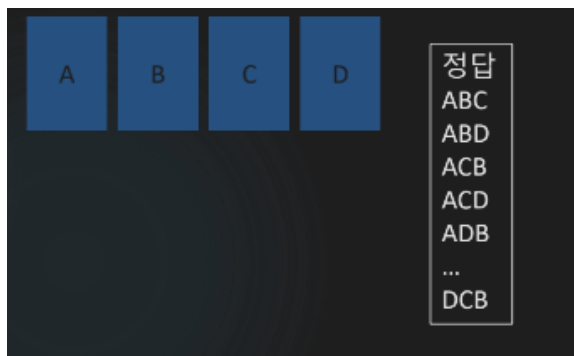
"AA?" 패턴 시도:

└─ Level 0: A 저장

└─ Level 1: A 저장

└─ Level 2: 진입 → level>=2 조건 체크 → path[0]=='A', path[1]=='A'
→ 같으므로 return으로 즉시 종료!

내장의 카드 중 한번 뽑았던 카드는 사다 안뽑고, 세장만 뽑는 경우



중복 없는 조합 만들기 - visited 배열 활용

카드가 사용 되었는지 사용 되지 않았는지 중복체크 해주기 위한 배열 visited을 만들어서 체크를 해준다.

```
#include <iostream>
```

```
char path[5] = "";
```

```
int visited[5] = {}; // 📇 카드 사용 여부 체크 (0: 미사용, 1: 사용중)
```

```
void test(int level)
```

```
{
```

```
    if (level == 3)
```

```
    {
```

```
        std::cout << path << std::endl;
```

```
        return;
```

```
    }
```

```
    for (size_t i = 0; i < 4; i++) // A, B, C, D 4장 중에서
```

```
    {
```

```
        // 💡 이미 사용중인 카드면 건너뛰기
```

```
        if (visited[i] == 1)
```

```
            continue;
```

```
        // 📌 카드 사용 시작
```

```
        visited[i] = 1; // 사용 표시
```

```
        path[level] = 'A' + i; // 경로에 저장
```

```

    test(level + 1);    // 다음 레벨로

    // 🔄 백트래킹: 원상복구 (다른 조합을 위해)
    path[level] = 0;    // 경로에서 제거
    visited[i] = 0;     // 사용 해제
}
}

```

☁️ visited 배열 동작 원리:

초기: visited = [0, 0, 0, 0] (A, B, C, D 모두 사용 가능)

Level 0에서 A 선택:

- └─ visited = [1, 0, 0, 0] ← A 사용중 표시
- └─ Level 1에서 B 선택: visited = [1, 1, 0, 0]
- └─ Level 2에서 C 선택: visited = [1, 1, 1, 0] → "ABC" 완성!
- └─ 백트래킹으로 Level 1로 돌아가며: visited = [1, 0, 0, 0]
- └─ Level 1에서 C 선택: visited = [1, 0, 1, 0] → "ACB" 시도...

🎯 핵심 포인트:

- **방법1:** 애초에 안 들어가게 차단 (`continue`)
- **방법2:** 들어갔다가 조건 맞으면 나가기 (`return`)
- **visited:** 중복 사용 방지를 위한 상태 관리 배열

"강의는 많은데, 왜 나는 아직도 코드를 못 짤까?"

혼자 공부하다 보면 누구나 이런 고민을 하게 됩니다.

- 강의는 다 들었지만 막상 **손이 안 움직이고**,
- 복습을 하려 해도 **무엇을 다시 봐야 할지 모르겠고**,
- 질문할 곳도 없고,
- 유튜브는 결국 **정답을 따라 치는 것밖에 안 되는 것 같고**.

문제는 '**연습**'이 빠졌기 때문입니다.

단순히 강의를 듣는 것만으로는 실력이 늘지 않습니다.

실제 문제를 풀고, 고민하고, 직접 구현해보는 시간이 반드시 필요합니다.

그래서, 얀얀코딩 코칭은 다릅니다.

그냥 가르치지 않습니다.

스스로 설계하고, 코딩할 수 있게 만듭니다.

얀얀코딩 코칭에서는 단순한 예제가 아닌,

스스로 문제를 분석하고 구현해야 하는 연습문제를 제공합니다.

이 연습문제들은 다음과 같은 역량을 키우기 위해 설계되어 있습니다:

- 문제를 스스로 쪼개고 설계하는 힘
- 다양한 조건을 만족시키는 실제 구현 능력
- 기능 단위가 아닌, 프로그램 단위로 사고하는 습관
- 마침내 자신의 힘으로 코드를 끝까지 작성하는 경험

지금 필요한 건 더 많은 강의가 아닙니다.

코드를 스스로 완성해 나가는 훈련,

그것이 지금 실력을 끌어올릴 가장 현실적인 방법입니다.

👉 자세한 안내 보기: [프리미엄 코칭 안내 바로가기](#)

👉 또는 카카오톡 상담방: [얀얀코딩 상담방](#)

▼ oopy:hide

연습 문제



1등, 2등, 3등 선물주기

문제 1번

네 명의 친구들의 이름을 입력 받아 주세요. (사람당 한글자씩, 총 4글자)

그리고 이 친구들 중 1등, 2등, 3등을 뽑아 선물을 주려고 합니다.

한 사람은 하나의 선물만 받을 수 있습니다.

선물을 줄 수 있는 경우를 모두 출력 해주세요.

ex)

[입력]

ATKP

[출력]

ATK

ATP

AKT

AKP

APT

APK

TAK

TAP

TKA

TKP

TPA

TPK

KAT

KAP

KTA

KTP

KPA

KPT

PAT

PAK

PTA

PTK

PKA

PKT

[TIP : 중복순열과 순열의 차이]

A,B,C,D 중 2장을 뽑는 경우의 수

중복순열 : 16가지 (AA, AB, AC, AD / BA, BB, BC, BD / CA, CB, CC, CD / DA, DB, DC, DB)

순열 : 4가지 (AB, AC, AD / BA, BC, BD / CA, CB, CD / DA, DB, DC)

입력 예제

ATKP

출력 결과

ATK

ATP

AKT

AKP

APT

APK

TAK

TAP

TKA

TKP

TPA

TPK

KAT

KAP

KTA

KTP

KPA

KPT

PAT

PAK

PTA

PTK

PKA

PKT



다툼친구 B와 T

문제 2번

네 글자를 입력 받으세요.

네 글자를 조합하여 나올수 있는 모든 경우가 몇가지인지 알아내고자 합니다.

그런데 B와 T 글자는 서로 붙어있으면 안됩니다.

재귀호출을 이용해서 풀어주세요

ex)

만약, BOTK 네글자를 입력받았다면,

BBBB → 가능

BBBT → 불가능

BOOT → 가능

TTBK → 불가능

TTTK → 가능

네 글자를 입력받고,

B와 T글자가 서로 붙어있지 않은 총 경우의 수가 몇 가지인지 출력하세요.

입력 예제

BOTT

출력 결과

120



ABC초콜릿

문제 3번

A / B / C 세 종류의 초콜릿이 있습니다

이 중 n 개의 초콜릿을 선택하려고 하는데

3개 연속으로 같은 알파벳의 초콜릿을 선택하면 안됩니다.

가져갈 n 개의 초콜릿 개수를 입력받고, 나올수 있는 총 가짓수를 출력해주세요.

(재귀호출을 이용해서 풀어주세요)

ex) 3개의 초콜릿을 선택한다고 한다면

AAA → 불가능

AAB → 가능

AAC → 가능

ABA → 가능

...

CCC → 불가능

• -----

총 24가지

입력 예제

3

출력 결과

24



산타소년단

문제 4번

인기그룹 산타소년단이 있습니다.

멤버 다섯명의 이름은 각각 B T S K R 입니다.

이 그룹에서 n 명을 순서대로 뽑아서 새로운 소규모 그룹을 만드려고 합니다.

첫 번째 뽑인사람이 리더이며,

두 번째 이후부터는 세컨드, 서드 등등 역할이 주어집니다.

멤버 중 방송국 국장 아들인 S군은 새로운 소규모 팀에 들어있어야 합니다.

n 을 입력받으세요.

그리고 n 명을 뽑아 멤버를 구성하려고 할 때,

나올 수 있는 순열의 수를 Counting해서 출력 해 주세요.

ex) 3

BTS

BST

BSK

BSR

BKS

BRS

TBS

TSB

TSK

TSR

TKS

TRS

SBT

SBK

SBR

STB

STK

STR

SKB

SKT

SKR

SRB

SRT

SRK

KBS

KTS

KSB
KST
KSR
KRS
RBS
RTS
RSB
RST
RSK
RKS

총 36개

[힌트]

cnt++ 하기 전, via['S'의 index] == 1 인지 확인하면

S가 포함되었는지 알 수 있음

입력 예제

3

출력 결과

36



미안하다 친구야

문제 5번

'E', 'W', 'A', 'B', 'C' 라는 친구들이 놀이기구를 타려고 합니다.

이 놀이기구는 가장 앞좌석부터 뒷좌석까지 4명이 탈 수 있는 보트입니다.

다섯명의 친구들 중 탈 순서를 정해야 하는데,

한명을 제외시켜야 합니다.

제외시킬 친구를 입력받고 (문자 1개입력)

이 친구를 제외한 모든 순열을 출력 해 주세요.

입력 예제

E

출력 결과

WABC

WACB

WBAC

WBCA

WCAB

WCBA

AWBC

AWCB

ABWC

ABCW

ACWB

ACBW

BWAC

BWCA

BAWC

BACW

BCWA

BCAW

CWAB

CWBA

CAWB

CABW

CBWA

CBAW



다섯종류의 숫자카드

문제 6번

다섯 종류의 카드를 입력받습니다. ('0' ~ '9')

각각의 카드들은 다량으로 쌓여있습니다.

다섯 종류의 숫자 카드에서 **4장**을 뽑으려고 합니다.

뽑을 때마다 전에 뽑았던 카드번호와 간격이 **3이하로** 차이나는

중복순열이 몇 가지인지 출력하세요.

재귀호출을 이용해서 풀어주세요

ex)

카드종류가 1/2/3/4/5 일때

1111 : OK

1112 : OK

1113 : OK

1114 : OK

1115 : [NO]

1121 : OK

...

no count 숫자 : 1251, 5123 .. 등등

• -----

총 497가지

[힌트]

path[level - 1] 와 path[level] 의 절대값이 3 차이가 나는지 확인

입력 예제

12345

출력 결과

497

복습 문제



왼쪽, 오른쪽 이동
문제 1번

3	5	1	9	7
---	---	---	---	---

위 배열을 하드코딩 해주세요.

그리고 R 또는 L 문자 4개를 입력 받습니다.

R은 right 방향을 의미하고

L은 left 방향을 의미 합니다.

아래 그림과 같이

R을 입력 받으면 숫자를 오른쪽으로 한칸씩 모두 이동시키는데 맨 뒤에 있는 숫자는 맨앞으로 와야합니다.



반대로 L을 입력 받으면 숫자를 왼쪽으로 한칸씩 모두 이동 시키고 맨 앞에 있는 숫자는 맨 뒤로 보냅니다.



R 또는 L을 4번 입력 받은 후 처리된 결과를 출력 해주세요.

입력 예제

R

R

R

L

출력 결과

9 7 3 5 1



암살자 존획

문제 2번

#은 암살자들이 있는 위치입니다. 3명의 암살자의 위치를 입력 받으세요.
만약, 직선거리에 상대방이 있다면 서로 총을 쏘게 됩니다.

안전한 배치



#			
		#	
	#		

서로 총을 쏘는
위험한 배치



#			#
	#		

세명의 좌표를 입력 받고

서로 총을 쏘지 않는 안전한 위치라면 "안전" 출력

그렇지않다면 "위험"을 출력 해주세요.

입력 예제

0 0

1 2

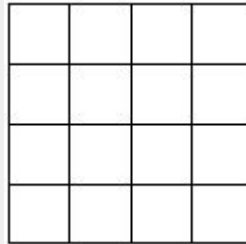
2 1

출력 결과

안전



네모네모 더하기 문제 3번



4×4 배열을 만들고 (0,0)~(2,2)까지 3 x 3칸에 다가 아홉 숫자를 입력 받으세요.

예를들어

1 2 1

2 3 4

3 2 1 을 입력 받았다면 아래와 같이 배열값이 넣어지게 됩니다.

그리고 빈칸에는 **가로줄의 합& 세로줄의 합 & 대각선줄의 합이 계산되어 채워 집니다.**

1	2	1	
2	3	4	
3	2	1	

→

1	2	1	4
2	3	4	9
3	2	1	6
6	7	6	5

배열에 모든 값이 채워지면 출력합니다.

적절한 for문을 사용하여 이 프로그램을 만들어 주세요

입력 예제

1 2 1

2 3 4

3 2 1

출력 결과

1 2 1 4

2 3 4 9

3 2 1 6

6 7 6 5



숫자 transformation 문제 4번

3	5	4	1
1	1	2	3
6	7	1	2

위 숫자들을 하드코딩 해주세요.

그리고 각기 다른 숫자 4개를 배열에 입력 받으세요.

예로들어 1 3 7 6 을 입력 받았다고 한다면, 이차배열의 값을

숫자 1을 3으로 변경

숫자 3을 7로 변경

숫자 7을 6으로 변경

숫자 6을 1로 변경

하시면 됩니다.

(이 외 나머지 숫자는 그대로 두시면 됩니다)

변경된 이차배열 값을 출력해주세요.

ex) 1 3 7 6 →

7	5	4	3
3	3	2	7
1	6	3	2

입력 예제

1 3 7 6

출력 결과

7 5 4 3

3 3 2 7

1 6 3 2



자기자리 찾기

문제 5번

8개의 숫자를 배열에 입력받아주세요.

배열에서 가장 왼쪽에 있는 숫자를 "피벗"이라고 합니다.

만약 아래와 같이 4 1 7 9 6 3 3 6을 입력받으면 피벗은 4가 됩니다.

4	1	7	9	6	3	3	6
---	---	---	---	---	---	---	---

이 배열을 가지고 아래에 나와있는 규칙대로 숫자들을 옮기다보면,
신기하게도

피벗 왼쪽에는 피벗보다 작은 숫자들이

피벗 오른쪽에는 피벗보다 큰 숫자들로 구성됩니다.

ex)

3	1	3	4	6	9	7	6
---	---	---	---	---	---	---	---

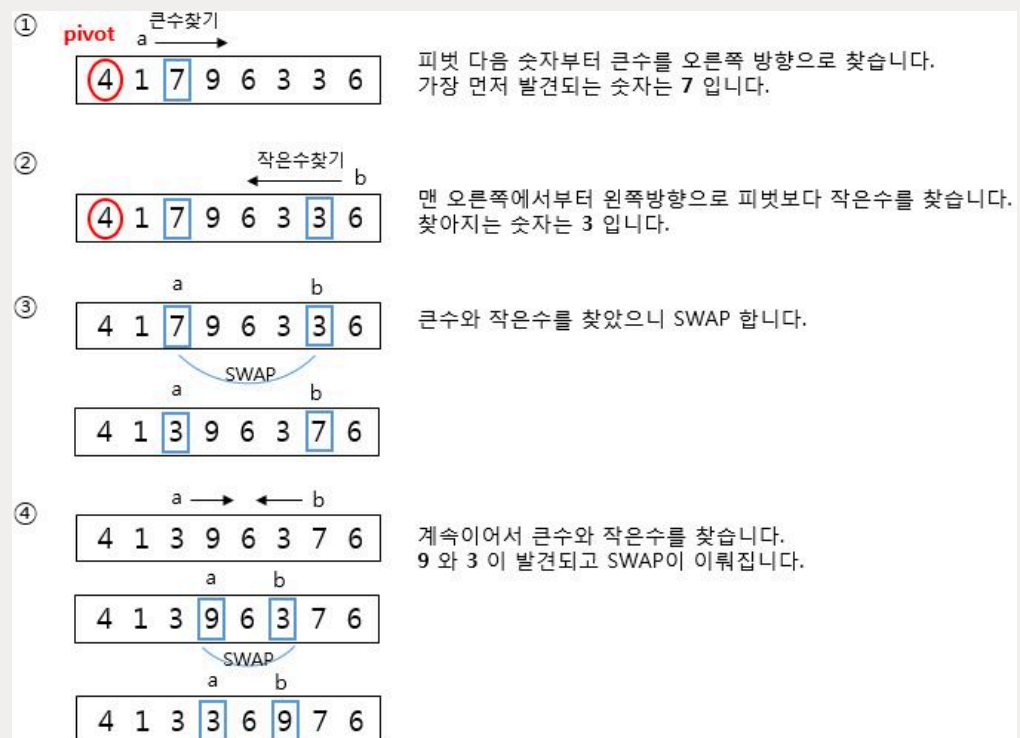
 적합한 위치로 이동 성공

아래의 규칙에 따라 숫자를 옮기고, 결과를 출력 해 주세요

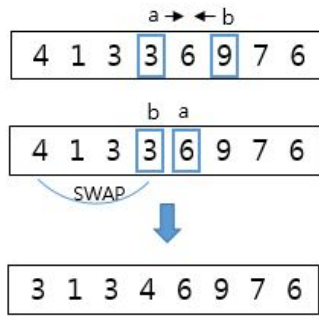
요약 :

a는 pivot보다 큰수를 찾아야하고

b는 pivot보다 작은수를 찾아서 SWAP 해야합니다.



⑤



계속이어서 큰수와 작은수를 찾는데 a, b가 엇갈렸습니다. 이때 b와 pivot을 교체하면 끝이 납니다.

완성

입력 예제

4 1 7 9 6 3 3 6

출력 결과

3 1 3 4 6 9 7 6



황금좌표 찾기

문제 6번

아래 그림을 보면 두개의 4x4 배열이 있습니다.

왼쪽배열 (4x4)는 입력받고,

오른쪽배열 (4x4)는 하드코딩 해 주세요.

이 두 배열에서 같은 좌표값이 같은 알파벳을 가지고 있으면 황금 좌표 입니다.

황금 좌표를 가장 많이 가진 알파벳을 찾아 출력 해주세요.

황금좌표

A	B	B	A
B	A	C	B
C	B	A	A
D	D	A	B

A	B	C	D
B	B	A	B
C	B	A	C
B	A	A	A

정답 : B

위 예제에서는 B가 황금좌표를 4개를 가지고 있기 때문에 정답은 B입니다.

입력 예제

ABBA

BACB

CBAA

DDAB

출력 결과

B